



Formation sur docker et les conteneurs OCI

 www.cwcloud.tech



contact@cwcloud.tech



CWCloud

229 rue Saint-Honoré, 75001, Paris

SIRET : 99220080800010





Au programme

❖ Les conteneurs

- différences entre VM et conteneur
- OCI open container initiative

❖ Docker

- Les notions de bases (images, layers, networks, volumes)
- Décomposition d'un Dockerfile
- Builder une image
- Démarrer un conteneur

❖ Docker compose

- A quoi ça sert ?
- Builder une image
- Démarrer des conteneurs

❖ Les registries publiques et privées

- docker hub
- harbor
- jfrog / artifactory
- Les autres
- Uploader une image sur une registry

❖ Analyser des images

- **diver** pour analyser le contenu des layers
- **trivy** pour analyser les vulnérabilités

❖ Cloud native friendly

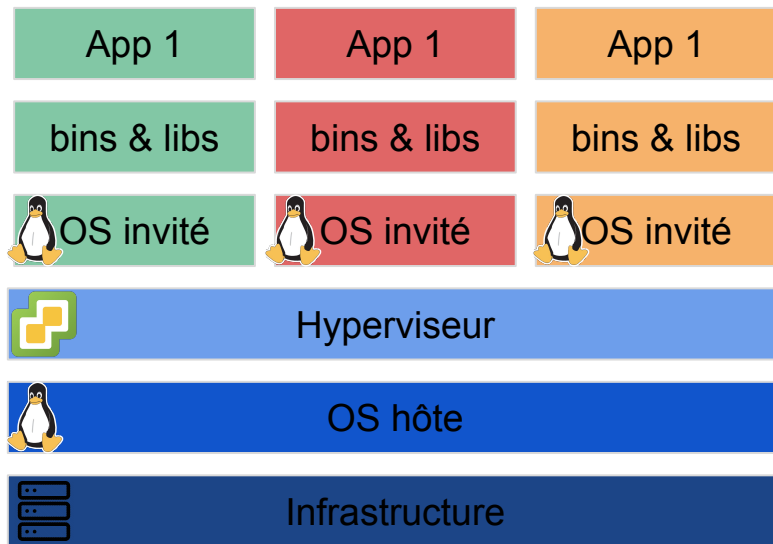
- Les bonnes pratiques
- Exemple en Java / Spring
- Exemple en Angular, React & co

❖ Mise en pratique

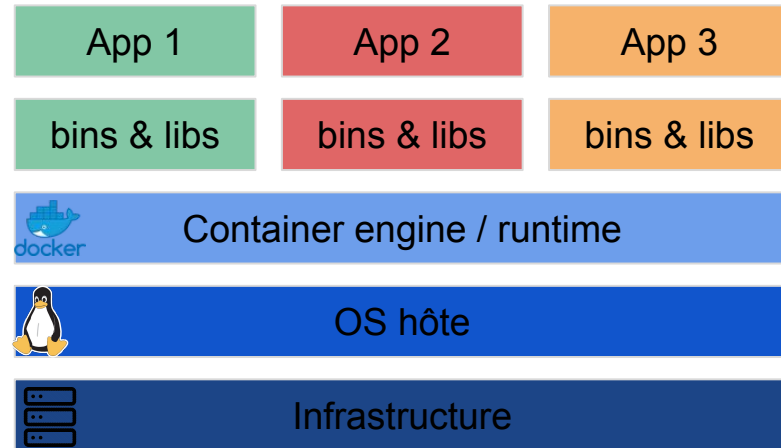
- conteneuriser une API en Python / flask
- conteneuriser une API en Java / Springboot
- conteneuriser une API en PHP / Lumen
- conteneuriser une application front Angular
- automatisation du build avec gitlab-ci

Les conteneurs OCI

Différences entre machines virtuelles et conteneurs



Machines virtuelles



Conteneurs

Les conteneurs OCI

Open container initiative (OCI)

Lancé en 2015 par Docker et d'autres leaders de l'industrie des conteneurs, il s'agit du standard permettant l'interopérabilité des images et conteneurs dits "OCI" : <https://opencontainers.org>

Aujourd'hui, il existe en effet aujourd'hui de nombreux runtimes de conteneurs (ou "containers engine"):

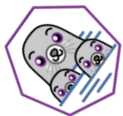
- containerd
- docker
- podman
- cri-o
- etc

Et différentes façon de construire des images:

- docker
- buildkit
- buildah
- kaniko
- etc



podman



cri-o



buildah



docker

OCI est le garant de l'interopérabilité des Dockerfiles et des images buildées d'une plateforme à l'autre pour être runnées d'une plateforme à l'autre.

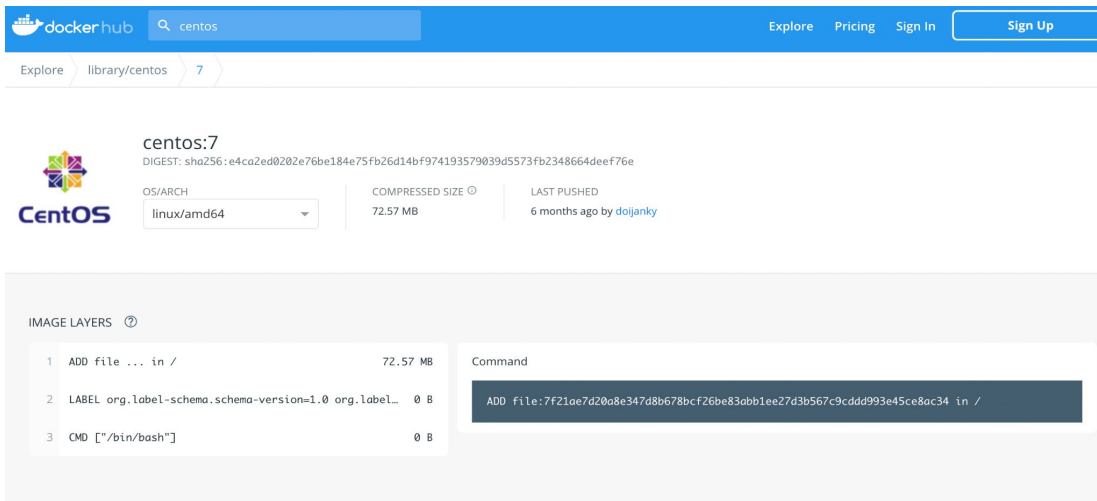
Docker

Les notions de bases

- ❖ Les images
 - identifiées par un nom et un tag
 - constituent un ensemble cohérent de “layers” (couches) pour démarrer un service atomique
 - peuvent hériter d'autres images ou ré-importer des layers d'autres images
 - les images de bases sont généralement des images publiques disponibles sur docker hub (ou autre registry publique)

```
lineumann on master * ~/docker $ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
harbor.comwork.io/todoapi/unittest	latest	54d886278da6	13 days ago	917MB
harbor.comwork.io/todoapi/api	latest	9f451edeb0d7	13 days ago	916MB
harbor.comwork.io/clockify/clockify	main-3f7c0792	74a24034adc5	2 weeks ago	51.6MB
harbor.comwork.io/clockify/clockify	main-9af2e6fd	448bdd0a4e97	2 weeks ago	10.5MB
harbor.comwork.io/clockify/clockify	latest	53a61c087072	3 weeks ago	10.9MB
harbor.comwork.io/mutual/mutual_ui	latest	8668758e129a	8 weeks ago	63.7MB
harbor.comwork.io/mutual/mutual_ws	latest	d1bc87d46858	8 weeks ago	305MB
docker.elastic.co/kibana/kibana	7.13.2	0d7fac58828d	8 weeks ago	1.51GB
harbor.comwork.io/hub/esf_test	latest	63d5447f3e60	2 months ago	880MB
tomcat	9	d9e19311a36b	2 months ago	658MB
openjdk	8	6b24bd100507	2 months ago	509MB
mutual_db	latest	ef5df7c9d93f	2 months ago	300MB
comworkio/cmd-api	latest	232195981de6	2 months ago	120MB
docker.elastic.co/elasticsearch/elasticsearch	7.6.1	41072cdeebc5	17 months ago	790MB
postgres	9.4	897a27671908	17 months ago	241MB



The screenshot shows the Docker Hub page for the `centos:7` image. The page includes the Docker Hub logo, a search bar, and navigation links. The main content area displays the image details for `centos:7`, including the digest, OS/ARCH (linux/amd64), compressed size (72.57 MB), and last pushed date (6 months ago by dojanky). Below this, the 'IMAGE LAYERS' section shows a list of layers with their sizes and the command used to create them.

Layer	Command	Size
1	ADD file ... in /	72.57 MB
2	LABEL org.label-schema.schema-version=1.0 org.label...	0 B
3	CMD ["/bin/bash"]	0 B

Docker

Les notions de bases



Les layers

- artifact OCI qui sont des sortes d'archives tar qui composent les images docker et correspondent au produit d'une instruction dans le Dockerfile
- les layers sont identifiés par un sha et peuvent être mutualisés pour différentes images (via l'héritage ou le multistage build avec **copy --from**)
- les layers permettent d'éviter d'avoir à être rebuildé ou retéléchargés sur l'hôte s'ils ne changent pas

```
ineumann on master * ~/docker $ docker inspect harbor.comwork.io/todoapi/unittest | jq .[0].RootFS
{
  "Type": "layers",
  "Layers": [
    "sha256:0c4db5d7ee48e8d916e6d1f6f6f77c8dbca383eb80ab74fa85b6911767523219",
    "sha256:b48bc43bef8b688ca2c18f93a382b4f3c362de52b7061d4c6048f02018b75c59",
    "sha256:665bd204ab72b3539803767ed3b49b63ea337a425c496fe1bd5cdeb782b9f7b8d",
    "sha256:4859da74ce517234d0198363f169e43c7d8f005d4f2729b4300b4823d1d8c6d9",
    "sha256:410ec4a217374a1cea90fa0d91756571e0d6c380186b974b4c22bda54d0716f",
    "sha256:f876c0b805f9da088e2613342822b90fafc891c5b27e5f62432fa4f0035ac5da",
    "sha256:556a8c5d4e82079e79b80d5f1abdbdb44c093021415c799d9bbc60a4f3ab7f7",
    "sha256:9a9341f9cdf4e2f99768c0a1517eaa3416ae128acea5875d1642bcc9efafc7cb",
    "sha256:bba7cdea55f941b1bc7ad681d2c4b577807e70ddb733a82b414f385f20c7c976",
    "sha256:679568fa6490a2c09068a9410ad252549ecc7ed112df0b39d9b65ae689c394ca",
    "sha256:5f70bf18a08607016e948b04aed3b82103a36bea41755b6cddfaf10ace3c6ef",
    "sha256:f70ffdd13c9eead068cba7d9b9fcdff551d2026eedc8a2f99a493fd7bc65d8541",
    "sha256:5f70bf18a08607016e948b04aed3b82103a36bea41755b6cddfaf10ace3c6ef",
    "sha256:ab1a79ebfa310d77ae0f8176791b945c4e886e993d600702fb46f6eb9da8e26ee"
  ]
}
```

```
ineumann on master * ~/docker $ docker pull docker.elastic.co/elasticsearch/elasticsearch:7.11.0
7.11.0: Pulling from elasticsearch/elasticsearch
0122c235edee: Already exists
9bc50b2741f6: Downloading [=====] 15.29MB/24.5MB
4697480b6de2: Download complete
add2fd0c5df: Downloading [=] 10.77MB/346.5MB
36f20916e73d: Download complete
2fd6f9204a99: Download complete
cb1cc36d3a3f: Download complete
```

Docker

Les notions de bases



Les networks

- permettent en fonction du driver de faire communiquer les conteneurs entre eux sur des réseaux virtuels (bridge) ou partager l'interface réseau de l'hôte (host) ou bien d'être complètement étanche (none)

```
ineumann on master * ~/docker $ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
def4764dba20	bridge	bridge	local
517952c7841c	host	host	local
11d5c2942d3e	none	null	local
edb0c2b6b5a9	pipeline_api_default	bridge	local
9644f1818d40	talend-docker-v2_default	bridge	local
135c0393ba81	todoapi_default	bridge	local
1a045c7435dc	todoapi_todo_api	bridge	local

```
ineumann ~ $ docker network inspect pipeline_api_default
[
  {
    "Name": "pipeline_api_default",
    "Id": "edb0c2b6b5a96401338384619b6745732e609828db1a9d3cb311c95a3da5fa35",
    "Created": "2021-07-30T11:15:57.853188176Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.20.0.0/16",
          "Gateway": "172.20.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": true,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {},
    "Labels": {
      "com.docker.compose.network": "default",
      "com.docker.compose.project": "pipeline_api",
      "com.docker.compose.version": "1.29.2"
    }
  }
]
ineumann ~ $ docker network inspect host
[
  {
    "Name": "host",
    "Id": "517952c7841c6a31ddf27577de9107167149de64f80537ed85968bf6b6ac3566",
    "Created": "2021-06-04T09:28:37.435149835Z",
    "Scope": "local",
    "Driver": "host",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": []
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {},
    "Labels": {}
  }
]
```

Docker

Les notions de bases



Les volumes

- permettent d'assurer la persistance des données d'un conteneur même dans le cas où il est amené à être détruit et reconstruit
- permet de monter un répertoire ou fichier de l'hôte à l'intérieur d'un conteneur (ainsi il est par exemple possible d'utiliser des conteneurs génériques comme un openjdk par exemple pour démarrer n'importe quel fichier jar qu'on aurait construit sur l'hôte local)

```
ineumann on master * ~/docker $ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
ad3aa3fb2368   postgres:9.4  "docker-entrypoint.s..."  4 days ago    Up 4 days    0.0.0.0:5436->5432/tcp, :::5436->5432/tcp  todo_db

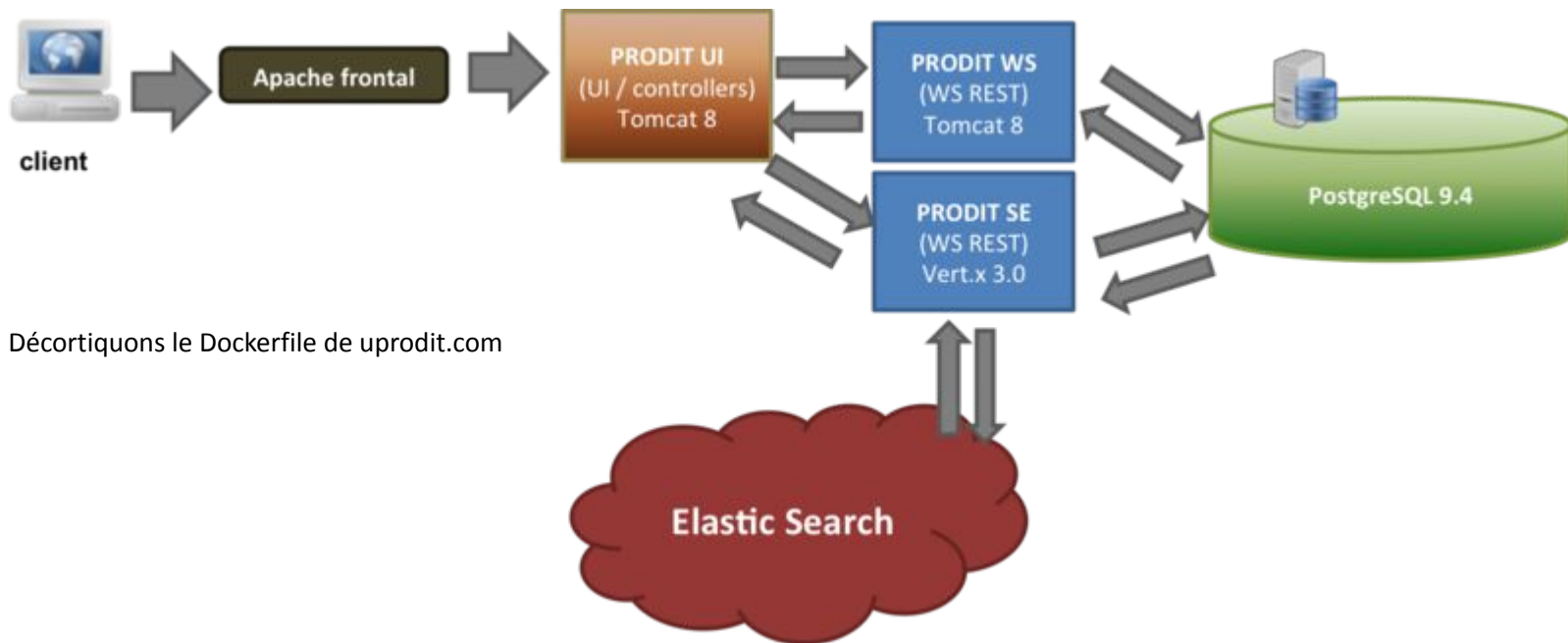
ineumann on master * ~/docker $ docker inspect todo_db|jq .[0].Mounts
[
  {
    "Type": "bind",
    "Source": "/Users/ineumann/docker/todoapi/install.sql",
    "Destination": "/install.sql",
    "Mode": "ro",
    "RW": false,
    "Propagation": "rprivate"
  },
  {
    "Type": "bind",
    "Source": "/Users/ineumann/docker/todoapi/data_volume",
    "Destination": "/var/lib/postgresql/data",
    "Mode": "z",
    "RW": true,
    "Propagation": "rprivate"
  }
]
```

```
ineumann on master * ~/uprodit $ docker volume ls
DRIVER      VOLUME NAME
local       1f88a57a84e8884cc1cb3a1aebd7cbde4858e939e2c4317aca969743d17531bc
local       6978f2b8c520158d742d24a46bc012822d3f65b84d8f2b0eff8d1dadeec5e1f8
local       ad01b709717894072317e37570a584116300da877c80a8f9cb336c21c8629aba
local       b488bf70c8522aeb2335a8df37c6e12af9d60e1dfdaae06434fc9846c89fdff8
local       c1c4bf057f69e6838797d4238bf20117ba7d8ec0edc49da8851fca757ed8835
local       c04858547e36b36f8e05b240fbd923796feee64966436b6a97141d349ae3669
local       e7ecbd2f2614870a70866d6dbdc1e62cd685fd26aca67d58157a3124773fa8d22

ineumann on master * ~/uprodit $ docker volume inspect 1f88a57a84e8884cc1cb3a1aebd7cbde4858e939e2c4317aca969743d17531bc
[
  {
    "CreatedAt": "2021-07-03T10:38:14Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/1f88a57a84e8884cc1cb3a1aebd7cbde4858e939e2c4317aca969743d17531bc/_data",
    "Name": "1f88a57a84e8884cc1cb3a1aebd7cbde4858e939e2c4317aca969743d17531bc",
    "Options": null,
    "Scope": "local"
  }
]
```


Docker

Décomposition d'un Dockerfile



Décortiquons le Dockerfile de uprodit.com

Docker

Décomposition d'un Dockerfile

```
ARG MAVEN_VERSION=3.3-jdk-8
ARG TOMCAT_VERSION=9.0-jre8-slim
ARG OPENJDK_VERSION=8-jre-slim
```

Arguments de build (que l'on peut surcharger mais qui ont des valeurs par défauts qu'on pourra ré-utiliser comme variable au moment du build)

```
#####
# Stage spring_service_build: build maven for spring base app #
#####
```

```
FROM maven:${MAVEN_VERSION} AS spring_service_build
```

Build multistage (images temporaires dont on va se servir pour exporter certains layers résultant du build de ces stages)

```
ARG IMAGE_SERVICE
ENV BASE_DIR="/uprodit"
ARG MVN_ARTIFACT_VERSION=2.0.1-SNAPSHOT
```

Image de base (accessible depuis dockerhub) sur laquelle on va hériter cette image de stage

```
WORKDIR ${BASE_DIR}
```

```
COPY . ${BASE_DIR}
```

Argument permettant de définir le profil maven de l'application à builder (évite de devoir dupliquer toutes ces stages pour chacune des applications java à construire : on peut garder les mêmes stages pour 150 images comme pour une seule)

```
COPY ./prodit-batch/src/main/resources/env/docker/tomcat/log4j2.xml /
```

```
RUN mv /log4j2.xml /uprodit/prodit-${IMAGE_SERVICE}/src/main/resources && \
    mvn clean install -Dmaven.test.skip -P ${IMAGE_SERVICE} && \
    mv /uprodit/prodit-${IMAGE_SERVICE}/target/prodit-${IMAGE_SERVICE}-${MVN_ARTIFACT_VERSION}.war /ROOT.war && \
    mv manifest.json /manifest.json
```

Docker

Décomposition d'un Dockerfile

```
ARG MAVEN_VERSION=3.3-jdk-8
ARG TOMCAT_VERSION=9.0-jre8-slim
ARG OPENJDK_VERSION=8-jre-slim
```

Arguments de build (que l'on peut surcharger mais qui ont des valeurs par défauts qu'on pourra ré-utiliser comme variable au moment du build)

```
#####
# Stage spring_service_build: build maven for spring base app #
#####
```

```
FROM maven:${MAVEN_VERSION} AS spring_service_build
```

Build multistage (images temporaires dont on va se servir pour exporter certains layers résultant du build de ces stages)

```
ARG IMAGE_SERVICE
ENV BASE_DIR="/uprodit"
ARG MVN_ARTIFACT_VERSION=2.0.1-SNAPSHOT
```

Image de base (accessible depuis dockerhub) sur laquelle on va hériter cette image de stage

```
WORKDIR ${BASE_DIR}
```

```
COPY . ${BASE_DIR}
```

Argument permettant de définir le profil maven de l'application à builder (évite de devoir dupliquer toutes ces stages pour chacune des applications java à construire : on peut garder les mêmes stages pour 150 images comme pour une seule)

```
COPY ./prodit-batch/src/main/resources/env/docker/tomcat/log4j2.xml /
```

```
RUN mv /log4j2.xml /uprodit/prodit-${IMAGE_SERVICE}/src/main/resources && \
    mvn clean install -Dmaven.test.skip -P ${IMAGE_SERVICE} && \
    mv /uprodit/prodit-${IMAGE_SERVICE}/target/prodit-${IMAGE_SERVICE}-${MVN_ARTIFACT_VERSION}.war /ROOT.war && \
    mv manifest.json /manifest.json
```

Docker

Décomposition d'un Dockerfile

```
#####
# Stage vertx_service_build: build maven for vert.x base app #
#####
```

```
FROM maven:${MAVEN_VERSION} AS vertx_service_build
```

```
ARG IMAGE_SERVICE
```

```
ENV BASE_DIR="/uprodit"
```

```
ARG MVN_ARTIFACT_VERSION=0.0.1-SNAPSHOT
```

```
WORKDIR ${BASE_DIR}
```

```
COPY . ${BASE_DIR}
```

```
COPY ./prodit-batch/src/main/resources/env/docker/vertx/log4j2.xml /
```

```
RUN mv /log4j2.xml /uprodit/prodit-${IMAGE_SERVICE}/src/main/resources && \
    mvn clean install -Dmaven.test.skip -P ${IMAGE_SERVICE} && \
    mv /uprodit/prodit-${IMAGE_SERVICE}/target/prodit-${IMAGE_SERVICE}-${MVN_ARTIFACT_VERSION}-fat.jar /vertx-fat.jar && \
    mv /uprodit/prodit-${IMAGE_SERVICE}/src/main/resources/config.json /config.json && \
    mv manifest.json /manifest.json
```

Fixer le répertoire dans lequel on va effectuer les commandes de build (RUN)

copie de fichier ou répertoires comme layers à l'intérieur de l'image au moment du build (à pas confondre avec un volume non immutable monté au runtime)

Commandes qui sont effectuées au moment du build de l'image (et non pas du runtime du conteneur). Il faut éviter de répéter les instructions RUN et les condenser en une seule pour optimiser le nombre de layers immutables

Docker

Décomposition d'un Dockerfile

```
#####  
# Stage spring_service: exposing a spring based app #  
#####
```

```
FROM tomcat:${TOMCAT_VERSION} AS spring_service
```

```
COPY ./prodit-batch/src/main/resources/env/docker/tomcat/server.xml /usr/local/tomcat/conf/server.xml:z  
COPY ./prodit-batch/src/main/resources/env/docker/tomcat/catalina.sh /usr/local/tomcat/bin/catalina.sh:z  
COPY ./prodit-batch/src/main/resources/env/docker/tomcat/logging.properties  
/usr/local/tomcat/conf/logging.properties:z
```

```
RUN mkdir -p /prodit/prodit_cache /BACK_PRODIT && \
```

```
rm -rf /usr/local/tomcat/webapps/ROOT && \  
rm -rf /usr/local/tomcat/webapps/manager && \  
rm -rf /usr/local/tomcat/webapps/examples && \  
rm -rf /usr/local/tomcat/webapps/host-manager && \  
rm -rf /usr/local/tomcat/webapps/docs
```

Bonne pratique numéro 1 : on nettoie tout ce qui ne sert plus à rien (issu de l'image de base héritée). Cela prendra moins de place et surtout réduira la surface d'attaque du conteneur

```
COPY --from=spring_service_build /ROOT.war /usr/local/tomcat/webapps/ROOT.war  
COPY --from=spring_service_build /manifest.json /manifest.json
```

```
EXPOSE 8080
```

```
ENTRYPOINT ["catalina.sh", "run"]
```

Bonne pratique numéro 2 : copie de fichiers en provenance de layers construit par les stages précédentes (from={nom de la stage}). Permet de ne prendre que ce qui sera utile au runtime

Docker

Décomposition d'un Dockerfile

```
#####  
# Stage vertx_service: exposing a vertx based app #  
#####
```

```
FROM openjdk:${OPENJDK_VERSION} AS vertx_service
```

```
ENV VERTX_PORT=80
```

```
COPY --from=vertx_service_build /vertx-fat.jar /config.json /manifest.json /
```

```
EXPOSE 80
```

➔ Port qui sera exposé à l'instanciation du conteneur sur le réseau docker

```
CMD ["java", "-jar", "/vertx-fat.jar", "-conf", "/config.json"]
```

➔ Commande qui sera exécutée au runtime du conteneur

Docker

Construire une image

```
ineumann ~ $ docker build . harbor.comwork.io/prodit/prodit-se:latest --build-arg IMAGE_SERVICE=se --target vtx_service
```

Context de build
(arborescence dans laquelle
on va exécuter les copies, etc)

Nom de l'image (sera utilisée
pour pusher sur une registry, si
pas de domaine précisé =>
dockerhub)

On set l'argument de build
avec le nom du profil maven
de l'application pour laquelle
on veut construire une image

Nom de la stage
générique qui va nous
servir à builder l'image
finale (les stages
intermédiaires seront
automatiquement
rebuildés si besoin)

```
ineumann ~ $ DOCKER_BUILDKIT=1 docker build . harbor.comwork.io/prodit/prodit-se:latest --build-arg IMAGE_SERVICE=se --target vtx_service
```

Utiliser buildkit pour builder à la
place de docker (plus optimisé)

Docker

Démarrer un conteneur

```
ineumann ~ $ cat file.txt
```

Salut toi !

Fichier monté en volume au runtime (s'appellera target.txt à l'intérieur du conteneur mais il ne s'agira pas d'une copie)

```
ineumann ~ $ docker run -v
```

Salut toi !

~/file.txt:/target.txt

-it

ubuntu:latest

cat target.txt

Image utilisée pour créer le conteneur

Commande effectuée au démarrage du conteneur (si pas précisée => appliquer le bloque CMD de l'image de base)

Docker compose

A quoi ça sert ?

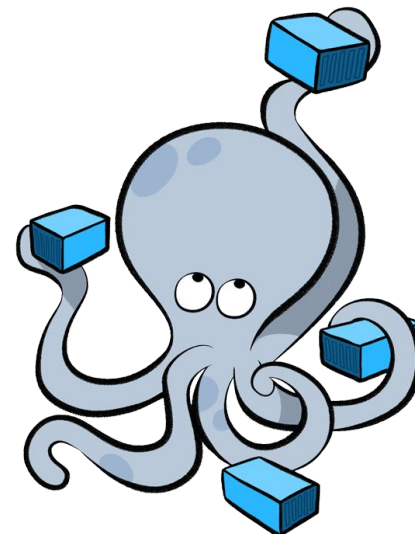
docker-compose permet de :

- orchestrer les images à builder
- orchestrer le démarrage d'un ensemble de conteneurs qui définissent une application

le tout au sein d'un même fichier ou d'un ensemble de fichier YAML structurés.

Jusqu'à récemment, docker-compose était une commande indépendante. Désormais une sous-commande "compose" est disponible dans la CLI de docker. Les deux sont encore disponibles mais il est désormais recommandé de partir sur la cli docker directement:

```
lineumann on master * ~/uprodit $ docker-compose -f docker-compose-local.yml up se
Creating network "uprodit_default" with the default driver
Creating prodit_elastic ... done
Creating se ... done
Attaching to se
se | 10:02:20.187 [vert.x-eventloop-thread-0] INFO tn.prodit.network.se.SearchEngineServer - Launching server...
se | 10:02:20.187 [vert.x-eventloop-thread-0] INFO tn.prodit.network.se.SearchEngineServer - Launching server...
se | Aug 08, 2021 10:02:20 AM io.vertx.core.impl.launcher.commands.VertxIsolatedDeployer
se | INFO: Succeeded in deploying verticle
^CGracefully stopping... (press Ctrl+C again to force)
Stopping se ... done
lineumann on master * ~/uprodit $ docker compose -f docker-compose-local.yml up se
[*] Running 2/2
  Container prodit_elastic Started 0.7s
  Container se Started 1.7s
Attaching to se
se | 10:02:33.191 [vert.x-eventloop-thread-0] INFO tn.prodit.network.se.SearchEngineServer - Launching server...
se | 10:02:33.191 [vert.x-eventloop-thread-0] INFO tn.prodit.network.se.SearchEngineServer - Launching server...
se | Aug 08, 2021 10:02:33 AM io.vertx.core.impl.launcher.commands.VertxIsolatedDeployer
se | INFO: Succeeded in deploying verticle
```



Docker compose

Builder une image

```
version: "3.8"

services:
  prodit_ws:
    image:
      harbor.comwork.io/prodit/prodit_ws:latest
    build:
      args:
        IMAGE_SERVICE: ws
      context: .
      dockerfile: ./Dockerfile
      target: spring_service
  prodit_se:
    image:
      harbor.comwork.io/prodit/prodit_se:latest
    build:
      args:
        IMAGE_SERVICE: se
      context: .
      dockerfile: ./Dockerfile
      target: vertx_service
```

```
ineumann $ docker compose -f docker-compose-build.yml build prodit_se
```

Utiliser buildkit pour builder à la
place de docker (plus optimisé)



```
ineumann $ COMPOSE_DOCKER_CLI_BUILD=1 DOCKER_BUILDKIT=1 docker compose -f
docker-compose-build.yml build prodit_se
```

Docker compose

Démarrer des conteneurs

```
version: "3.8"
```

```
services:
```

```
  prodit_elastic:
```

```
    image:
```

```
docker.elastic.co/elasticsearch/elasticsearch:7.9.1
```

```
    container_name: prodit_elastic
```

```
    environment:
```

- node.name=elasticsearch
- http.port=9200
- discovery.type=single-node
- cluster.name=elasticsearch
- bootstrap.memory_lock=true
- xpack.security.enabled=false
-

```
xpack.security.transport.ssl.enabled=false
```

- "ES_JAVA_OPTS=-Xms512m-Xmx512m"

```
    restart: always
```

```
    networks:
```

- prodit

```
  prodit_postgres:
```

```
    image: postgres:9.4
```

```
    container_name: prodit_postgres
```

```
    restart: always
```

```
    environment:
```

- POSTGRES_PASSWORD=prodit_webi
- POSTGRES_USER=prodit_webi
- POSTGRES_DB=prodit

```
    volumes:
```

```
      -  
      ./data_volume:/var/lib/postgresql/data  
:z
```

```
    networks:
```

- prodit

```
    ports:
```

- 5435:5432

```
  prodit_ui:
```

```
    image:
```

```
harbor.comwork.io/prodit/prodit_ui:${UPRODIT_VERSION}
```

```
    container_name: prodit-ui
```

```
    restart: always
```

```
    environment:
```

```
      CATALINA_OPTS:
```

```
-Droot.url=http://ppd.uprodit.com  
-Dse.service.url=http://prodit-se:80/prodit-se/api  
-Dws.service.url=http://prodit-ws:8080/api  
-Dws.statut.url=http://prodit-ws:8080/status -Dcache.redis.enabled=true  
-Dcache.redis.instance.host=prodit_redis:6379 -Dslack.channel=uprodit-ppd  
-Dslack.token=xoxb-...  
-Dsmtp.username=apikey  
-Dsmtp.host=smtp.sendgrid.net  
-Dsmtp.password=...
```

```
    networks:
```

- prodit

```
    ports:
```

- 8070:8080

Docker compose

Démarrer des conteneurs

```
prodit_ws:
  image:
    harbor.comwork.io/prodit/prodit_ws:${UPRODIT_VERSION}
  container_name: prodit-ws
  restart: always
  environment:
    CATALINA_OPTS: -Dlog.se.query=false
  -Dse.service.url=http://prodit-se:80/prodit-se/api
  networks:
    - prodit
  ports:
    - 8074:8080
```

```
prodit_se:
  image:
    harbor.comwork.io/prodit/prodit_se:${UPRODIT_VERSION}
  container_name: prodit-se
  restart: always
  networks:
    - prodit
  ports:
    - 8071:80

networks:
  prodit:
    driver: bridge
```

```
ineumann on master * ~/uprodit $ docker compose -f docker-compose-local.yml up -d se
```

```
[+] Running 2/2
```

```
:: Container prodit_elastic Started
```

```
:: Container se Started
```

Detached mode : détache le conteneur du processus courant et rend la main

```
ineumann on master * ~/uprodit $ docker compose -f docker-compose-local.yml up -d se --force-recreate --build
```

```
[+] Running 2/2
```

```
:: Container prodit_elastic Started
```

```
:: Container se Started
```

Force la recreation de l'image même si elle a déjà été buildée

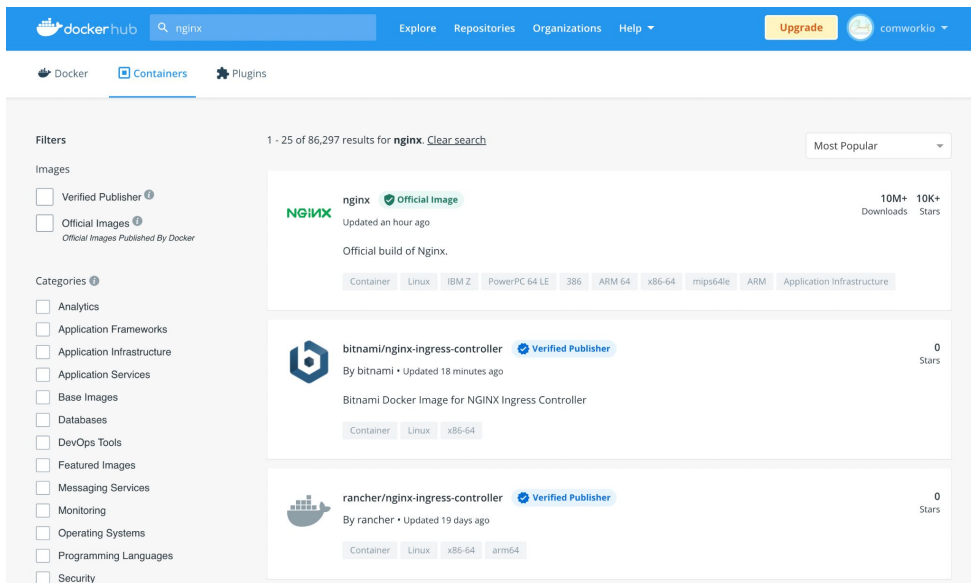
Re-crée le conteneur même s'il a déjà été démarré auparavant

```
ineumann on master * ~/uprodit $
```

Les registry publiques et privées

docker hub

Registry publique officielle de docker: <https://hub.docker.com>



The screenshot shows the Docker Hub interface. At the top, there's a navigation bar with 'dockerhub', a search bar containing 'nginx', and links for 'Explore', 'Repositories', 'Organizations', and 'Help'. Below the navigation bar, there are tabs for 'Docker', 'Containers', and 'Plugins'. The main content area displays search results for 'nginx'. On the left, there are filters for 'Images' (Verified Publisher, Official Images) and 'Categories' (Analytics, Application Frameworks, etc.). The search results show 1 - 25 of 86,297 results. The top result is 'nginx' by 'nginx', marked as an 'Official Image' with 10M+ downloads and 10K+ stars. Below it are 'bitnami/nginx-ingress-controller' and 'rancher/nginx-ingress-controller', both marked as 'Verified Publisher'.

```
ineumann on master * ~/docker $ docker pull openjdk:15
15: Pulling from library/openjdk
4b26d50a9215: Downloading [>] 431.1kB/42MB
afc97fa40816: Downloading [==>] 589kB/14.03MB
035761e41be3: Downloading [>] 537.6kB/174.9MB
```

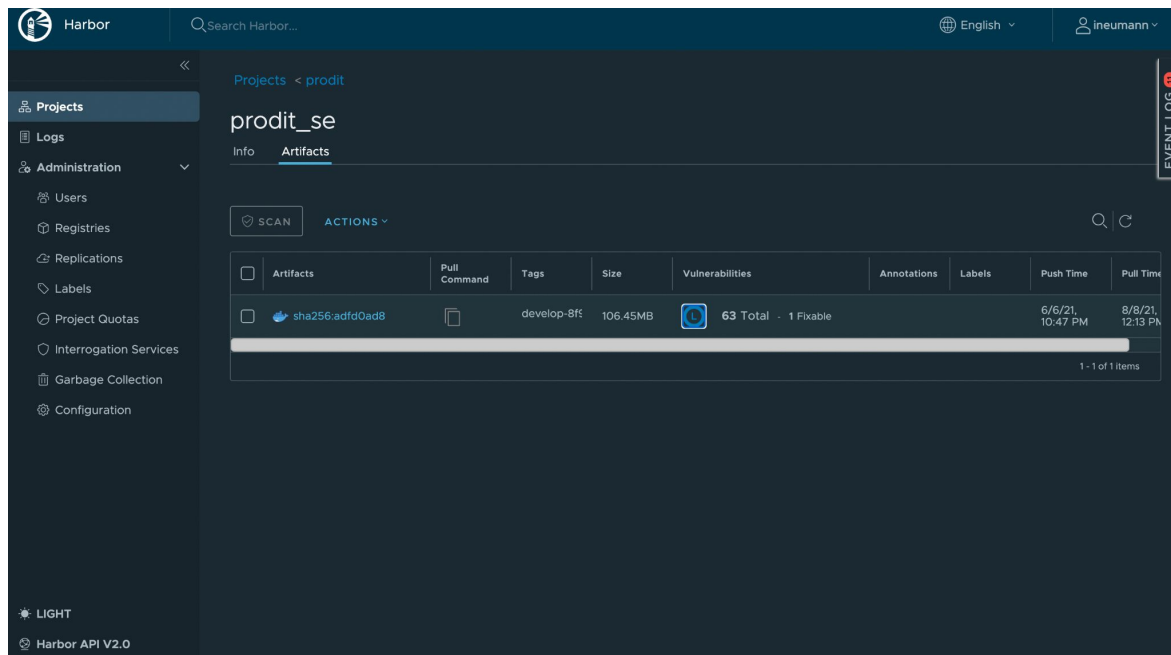
C'est la registry publique de référence et par défaut (les images pullées qui n'ont pas de nom de domaine de registry définie sont recherchées et téléchargées depuis le dockerhub)

Un abonnement payant permet également d'y uploader des images privées. Par défaut toutes les images sont en publique (adapté pour distribuer en opensource)

Les registry publiques et privées

harbor

Registry privée de référence de la CNCF (cloud native foundation): <https://goharbor.io>



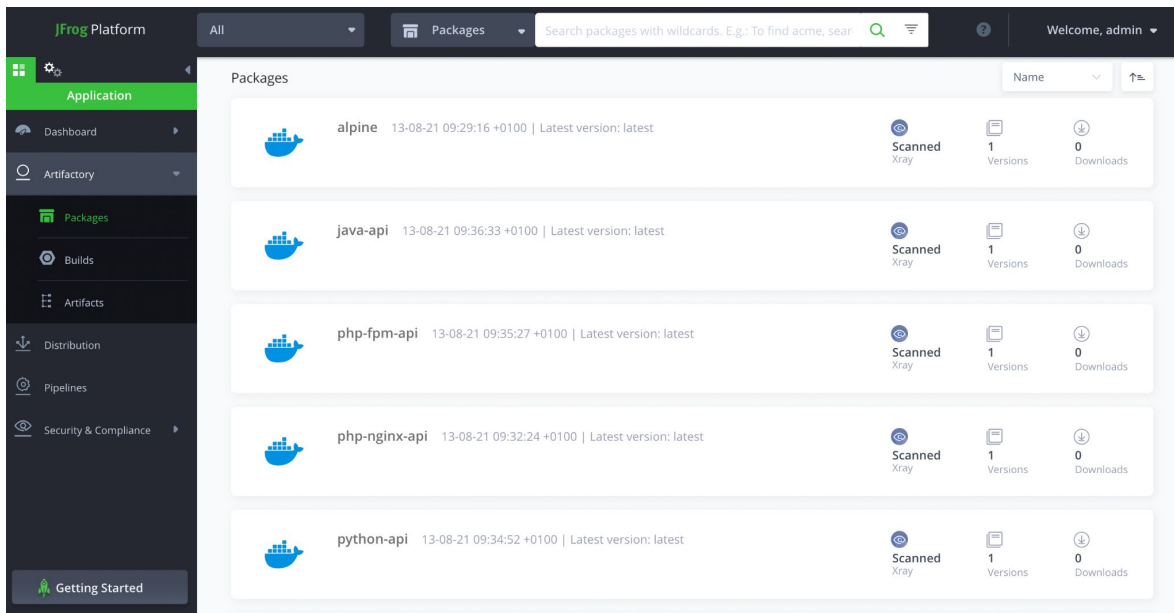
The screenshot shows the Harbor web interface. The left sidebar contains navigation links: Projects, Logs, Administration (Users, Registries, Replications, Labels, Project Quotas, Interrogation Services, Garbage Collection, Configuration), and LIGHT. The main content area displays the 'prodit_se' project page. The 'Artifacts' tab is selected, showing a table with columns: Artifacts, Pull Command, Tags, Size, Vulnerabilities, Annotations, Labels, Push Time, and Pull Time. A single artifact is listed with the ID 'sha256:adf0ad8', a pull command 'develop-8ft', a size of '106.45MB', and '63 Total - 1 Fixable' vulnerabilities. The push time is '6/6/21, 10:47 PM' and the pull time is '8/8/21, 12:13 PM'. The bottom of the interface shows 'Harbor API V2.0'.

- ❖ Opensource et self-hosted / on premise (partout ou il y a un OCI runtime en place)
- ❖ Permet à la fois d'héberger des repository d'images docker privées mais aussi de helm charts
- ❖ Scan des vulnérabilité sur les layers avec trivy embarqué
- ❖ Signature des images

Les registry publiques et privées

Jfrog / artifactory

Registry privée et payante de la plateforme Jfrog: <https://ifrog.com/>



Support de nombreux types de repos:

- registry docker
- helm charts
- python pip
- npm (typescript, javascript)
- php composer
- java (maven, graddle, etc)
- ruby
- go
- etc

De nombreuses autres features:

- scan de vulnérabilités avec xray
- pipelines CI/CD directement intégrées
- etc

Les registry publiques et privées

Les autres

Il en existe beaucoup d'autres:

- quay de RedHat: <https://quay.io>
- gitlab container registry: https://docs.gitlab.com/ee/user/packages/container_registry/
- Sonatype Nexus: <https://fr.sonatype.com/nexus/repository-oss>
- SaaS: github, google cloud platform, etc

Les registry publiques et privées

Uploader une image sur une registry

```
echo "${PASSWORD_OR_ACCESS_TOKEN}" | docker login --username "${USERNAME}" --password-stdin

docker tag "harbor.comwork.io/myrepo/my-image:latest" "harbor.comwork.io/myrepo/my-image:${branch}-${sha}"

docker push "harbor.comwork.io/myrepo/my-image:latest"
```

Analyser des images

dive pour analyser le contenu d'une image

Disponible ici: <https://github.com/wagoodman/dive>

ineumann on master * ~/docker \$ dive 74a24034adc5

		Current Layer Contents			
Command		Permission	UID:GID	Size	Filetree
FROM #f72598b05f57e6		-rwxr-xr-x	0:0	1.6 MB	bin
RUN /bin/sh -c apk add --no-cache postfix jq curl bash && apk add --no-cache --repository http://		-rwxr-xr-x	0:0	0 B	arch → /bin/busybox
COPY postfix /etc/postfix/ # buildkit		-rwxr-xr-x	0:0	0 B	ash → /bin/busybox
COPY mail/mailname /etc/mailname # buildkit		-rwxr-xr-x	0:0	0 B	base64 → /bin/busybox
COPY automate-result.sh . # buildkit		-rwxr-xr-x	0:0	736 kB	bash
		-rwxr-xr-x	0:0	0 B	bbconfig → /bin/busybox
		-rwxr-xr-x	0:0	841 kB	busybox
		-rwxr-xr-x	0:0	0 B	cat → /bin/busybox
		-rwxr-xr-x	0:0	0 B	chgrp → /bin/busybox
		-rwxr-xr-x	0:0	0 B	chmod → /bin/busybox
		-rwxr-xr-x	0:0	0 B	chown → /bin/busybox
		-rwxr-xr-x	0:0	0 B	conspy → /bin/busybox
		-rwxr-xr-x	0:0	0 B	cp → /bin/busybox
		-rwxr-xr-x	0:0	0 B	date → /bin/busybox
		-rwxr-xr-x	0:0	0 B	dd → /bin/busybox
		-rwxr-xr-x	0:0	0 B	df → /bin/busybox
		-rwxr-xr-x	0:0	0 B	dmesg → /bin/busybox
		-rwxr-xr-x	0:0	0 B	dnsdomainname → /bin/busybox
		-rwxr-xr-x	0:0	0 B	dumpmap → /bin/busybox
		-rwxr-xr-x	0:0	0 B	echo → /bin/busybox
		-rwxr-xr-x	0:0	0 B	ed → /bin/busybox
		-rwxr-xr-x	0:0	0 B	egrep → /bin/busybox
		-rwxr-xr-x	0:0	0 B	false → /bin/busybox
		-rwxr-xr-x	0:0	0 B	fatattr → /bin/busybox
		-rwxr-xr-x	0:0	0 B	fdflush → /bin/busybox
		-rwxr-xr-x	0:0	0 B	fgrep → /bin/busybox
		-rwxr-xr-x	0:0	0 B	fsync → /bin/busybox
		-rwxr-xr-x	0:0	0 B	getopt → /bin/busybox
		-rwxr-xr-x	0:0	0 B	grep → /bin/busybox
		-rwxr-xr-x	0:0	0 B	gunzip → /bin/busybox
		-rwxr-xr-x	0:0	0 B	gzip → /bin/busybox
		-rwxr-xr-x	0:0	0 B	hostname → /bin/busybox
		-rwxr-xr-x	0:0	0 B	ionice → /bin/busybox
		-rwxr-xr-x	0:0	0 B	iostat → /bin/busybox
		-rwxr-xr-x	0:0	0 B	ipcalc → /bin/busybox
		-rwxr-xr-x	0:0	0 B	kbd_mode → /bin/busybox
		-rwxr-xr-x	0:0	0 B	kill → /bin/busybox
		-rwxr-xr-x	0:0	0 B	link → /bin/busybox
		-rwxr-xr-x	0:0	0 B	linux32 → /bin/busybox
		-rwxr-xr-x	0:0	0 B	linux64 → /bin/busybox
		-rwxr-xr-x	0:0	0 B	ln → /bin/busybox
		-rwxr-xr-x	0:0	0 B	login → /bin/busybox
		-rwxr-xr-x	0:0	0 B	ls → /bin/busybox
		-rwxr-xr-x	0:0	0 B	lzo → /bin/busybox
		-rwxr-xr-x	0:0	0 B	makemime → /bin/busybox
		-rwxr-xr-x	0:0	0 B	mkdir → /bin/busybox
		-rwxr-xr-x	0:0	0 B	mknod → /bin/busybox
		-rwxr-xr-x	0:0	0 B	mktemp → /bin/busybox
		-rwxr-xr-x	0:0	0 B	more → /bin/busybox
		-rwxr-xr-x	0:0	0 B	mount → /bin/busybox
		-rwxr-xr-x	0:0	0 B	mountpoint → /bin/busybox
		-rwxr-xr-x	0:0	0 B	mpstat → /bin/busybox
		-rwxr-xr-x	0:0	0 B	mv → /bin/busybox
		-rwxr-xr-x	0:0	0 B	netstat → /bin/busybox
		-rwxr-xr-x	0:0	0 B	nice → /bin/busybox
		-rwxr-xr-x	0:0	0 B	pidof → /bin/busybox
		-rwxr-xr-x	0:0	0 B	ping → /bin/busybox
		-rwxr-xr-x	0:0	0 B	ping6 → /bin/busybox

Analyser des images

trivy pour analyser les vulnérabilités

```
ineumann on master * ~/docker $ trivy image harbor.comwork.io/clockify/clockify:main-3f7c0792
2021-08-08T16:51:28.122+0100 INFO Need to update DB
2021-08-08T16:51:28.122+0100 INFO Downloading DB...
22.77 MiB / 22.77 MiB [-----] 100.00% 1.11 MiB p/s 21s
2021-08-08T16:51:50.729+0100 INFO Detected OS: alpine
2021-08-08T16:51:50.729+0100 INFO Detecting Alpine vulnerabilities...
2021-08-08T16:51:50.732+0100 INFO Number of language-specific files: 0

harbor.comwork.io/clockify/clockify:main-3f7c0792 (alpine 3.12.1)
=====
Total: 26 (UNKNOWN: 1, LOW: 6, MEDIUM: 12, HIGH: 7, CRITICAL: 0)

+-----+-----+-----+-----+-----+-----+
| LIBRARY | VULNERABILITY ID | SEVERITY | INSTALLED VERSION | FIXED VERSION | TITLE |
+-----+-----+-----+-----+-----+-----+
| apk-tools | CVE-2021-30139 | HIGH | 2.10.5-r1 | 2.10.6-r0 | In Alpine Linux apk-tools before 2.12.5, the tarball parser allows a buffer... -->avd.aquasec.com/nvd/cve-2021-30139 |
+-----+-----+-----+-----+-----+-----+
| | CVE-2021-36159 | UNKNOWN | | 2.10.7-r0 | libfetch before 2021-07-26, as used in apk-tools, xbps, and other products, mishandles... -->avd.aquasec.com/nvd/cve-2021-36159 |
+-----+-----+-----+-----+-----+-----+
| busybox | CVE-2021-28831 | HIGH | 1.31.1-r19 | 1.31.1-r20 | busybox: invalid free or segmentation fault via malformed gzip data -->avd.aquasec.com/nvd/cve-2021-28831 |
+-----+-----+-----+-----+-----+-----+
| curl | CVE-2021-22922 | MEDIUM | 7.77.0-r0 | 7.78.0-r0 | curl: wrong content via metalink is not being discarded -->avd.aquasec.com/nvd/cve-2021-22922 |
+-----+-----+-----+-----+-----+-----+
| | CVE-2021-22923 | | | | curl: Metalink download sends credentials -->avd.aquasec.com/nvd/cve-2021-22923 |
+-----+-----+-----+-----+-----+-----+
| | CVE-2021-22924 | LOW | | | curl: bad connection reuse due to flawed path name checks -->avd.aquasec.com/nvd/cve-2021-22924 |
+-----+-----+-----+-----+-----+-----+
| | CVE-2021-22925 | | | | curl: Incorrect fix for CVE-2021-22898 TELNET stack contents disclosure -->avd.aquasec.com/nvd/cve-2021-22925 |
+-----+-----+-----+-----+-----+-----+
| libcrypto1.1 | CVE-2021-23840 | HIGH | 1.1.1g-r0 | 1.1.1j-r0 | openssl: integer overflow in CipherUpdate -->avd.aquasec.com/nvd/cve-2021-23840 |
+-----+-----+-----+-----+-----+-----+
| | CVE-2021-3450 | | | 1.1.1k-r0 | openssl: CA certificate check bypass with X509_V_FLAG_X509_STRICT -->avd.aquasec.com/nvd/cve-2021-3450 |
+-----+-----+-----+-----+-----+-----+
| | CVE-2020-1971 | MEDIUM | | 1.1.1i-r0 | openssl: EDIPARTYNAME NULL pointer de-reference -->avd.aquasec.com/nvd/cve-2020-1971 |
+-----+-----+-----+-----+-----+-----+
| | CVE-2021-23841 | | | 1.1.1j-r0 | openssl: NULL pointer dereference in X509_issuer_and_serial_hash() |
+-----+-----+-----+-----+-----+-----+
```

Disponible ici: <https://github.com/aquasecurity/trivy>

ineumann \$ trivy image
harbor.comwork.io/clockify/clockify:main-3f7c0792

Cloud native friendly

Les bonnes pratiques

Un conteneur cloud native friendly doit:

- être le plus stateless possible (ne pas grossir avec le temps, utiliser des volumes distribués tels que des PVC ou de buckets ou des bdds)
- écrire ses logs sur les sorties standards et erreurs standards (stdout, stderr)
- être complètement configurable via des variables d'environnements afin d'éviter de devoir re-constituer une image d'un environnement à l'autre et rester agnostic à ces environnements
- être observables :
 - avoir la possibilité de remonter des métriques (endpoints de métriques pour prometheus par exemple)
 - avoir des endpoints de healthcheck (les frameworks tels que SpringBoot ont déjà ce qu'il faut pour que ça soit facile à activer)

Cloud native friendly

Exemple en Java / Spring

```
@Configuration
public class PropertiesConfiguration {
    @Bean("properties")
    public Properties generateProperties() throws IOException {
        EnvironmentPropertiesFactoryBean factory = new EnvironmentPropertiesFactoryBean("application.properties");
        return factory.getObject();
    }
}
```

Bean à définir permettant de rendre l'application entièrement configurable avec des variables d'environnements. Si la variable d'environnement n'existe pas, on continue de chercher dans le fichier de configuration application.properties (utile quand on est sur une application déjà déployée en production en mode legacy qu'on voudrais déplacer sur du cloud type Kubernetes).

Cloud native friendly

Exemple en Java / Spring

```
public class EnvironmentPropertiesFactoryBean extends PropertiesFactoryBean {
    private static final Logger LOGGER = LogManager.getLogger(EnvironmentPropertiesFactoryBean.class);

    public EnvironmentPropertiesFactoryBean(String location) {
        this(new Resource[] {new ClassPathResource(location,
            EnvironmentPropertiesFactoryBean.class.getClassLoader())});
    }

    public EnvironmentPropertiesFactoryBean(Resource[] locations) {
        super.setIgnoreResourceNotFound(true);
        super.setLocations(locations);
    }

    private void setValueIfNotEmpty(Object key, Properties properties, String value) {
        if (isNotBlank(value)) {
            properties.put(key, StringEscapeUtils.unescapeHtml4(value));
        }
    }
}
```

Cloud native friendly

Exemple en Java / Spring

```
/**
 * Check if an environment variable exist for each keys of the merged properties files results if
 * this env var exists, replace the value of the property with its value
 */
private void overrideProperty(Properties properties, Object k) {
    String keyStr = k.toString();
    String envKey = keyStr.toUpperCase().replace(".", "_");

    setValueIfNotEmpty(k, properties, System.getenv(envKey));
    setValueIfNotEmpty(k, properties, System.getenv(keyStr));
    setValueIfNotEmpty(k, properties, System.getProperty(envKey));
    setValueIfNotEmpty(k, properties, System.getProperty(keyStr));
}

@Override
protected Properties mergeProperties() throws IOException {
    Properties result = super.mergeProperties();
    result.keySet().stream().forEach(k -> overrideProperty(result, k));
    return result;
}

@Override
protected void loadProperties(Properties props) throws IOException {
    super.loadProperties(props);
    props.keySet().stream().forEach(k -> overrideProperty(props, k));
}
}
```

Cloud native friendly

Exemple en Angular / React / Vue.js & co

```
#!/usr/bin/env sh
# vim:sw=4:ts=4:et

set -e

if [ "$1" = "nginx" ]; then
    defined_envs=$(printf '%s' ' $(env | cut -d= -f1))
    front_app_path="/ng-app-root"
    for file in $(find "${front_app_path}" -type f -name '*.css' -o -name '*.js' -o -name '*.html')
    do
        envsubst "$defined_envs" < $file > ${file}.tmp
        mv ${file}.tmp $file
    done

    # To prevent cache issues
    sed -i 's!\(vendor\|app\)\\.\\.([^\.]\\+\\.\\.\\(js\\|css\\)\\)!\\1.\\2.\\3?t='${date '+%s'}' '!g' "${front_app_path}/index.html"
fi

exec "$@"
```

Script à utiliser comme un entrypoint, il permet de rendre l'application "cloud native friendly", entièrement configurable avec des variables d'environnements. Il suffira ensuite de faire référence dans le code typescript ou fichiers de configuration à des variables telles que "\$WS_URL" qui seront dynamiquement remplacées par les variables d'environnement si elles sont définies, au démarrage du conteneur.

Mise en pratique

Bonnes pratiques dans l'écriture de Dockerfile pour différentes technologies

La suite se passe sur ce repo git (à cloner et suivre le README.md): https://gitlab.com/work.io/comwork_training/docker

